



RECRUITMENT ASSIGNMENT

Autonomous Platform Intelligence Agent

An engineering hiring assignment for candidates building autonomous, self-improving agents on real software platforms.

COMPANY	Watermelon Software Pte. Ltd.
VERSION	1.0
SUBMISSION	GitHub repository + 15-minute walkthrough and demo
CREATION DATE	19 Jun 2026



Confidential. This document contains proprietary information and is intended for authorized recipients only. Do not distribute without permission.

01 The Problem

Most SaaS platforms — workflow builders, project management tools, documentation systems — were designed for humans. A human reads the interface, decides what to do, clicks buttons, and builds things step by step. The moment you want to automate that work, you're writing brittle API wrappers or recording UI macros that break on every update.

The harder problem is not automation. It's **intelligent automation** — a system that can receive a natural language instruction, figure out how to execute it on a platform it has learned about, remember what it has done before, build new capabilities when it encounters something it cannot yet do, and get measurably better over time without being retrained.

That is what you are building.

02 The Assignment

Build an **Autonomous Platform Agent** for exactly one platform of your choice. The agent receives natural language instructions and executes them on the platform autonomously. It has persistent memory, learns from every execution, and can synthesise new capabilities at runtime when it encounters tasks it hasn't seen before.

03 Choose Your Platform

Pick one. It should be a platform where a human normally does meaningful, multi-step work:

- **n8n** — build, modify, and trigger workflows via API
- **Linear** — create issues, manage projects, triage backlogs, update roadmaps
- **Notion** — create pages, maintain databases, organise workspaces
- **GitHub** — manage issues, PRs, projects, releases
- **Airtable** — manage bases, create views, automate records

Pick the one where the automation problem is most interesting to you, not the one with the best API documentation.

04 What You Must Build

1 The Agent Core

An agent that accepts a natural language instruction and executes it on your chosen platform. The instructions can be simple ("create a bug report for the login timeout issue") or compound ("find all open issues assigned to nobody, group them by priority, and create a weekly triage summary page").

The agent must:

- Decompose compound instructions into executable steps
- Use the platform's API to execute each step
- Handle partial failures gracefully — if step 3 of 5 fails, it should not silently produce a half-complete result
- Return a structured execution report: what was done, what failed, why, and what the agent decided to do about it

2 Persistent Memory

The agent must have memory that survives across sessions. Memory is not a log file. It is structured knowledge the agent actively uses to make better decisions.

At minimum, your memory system must contain two distinct layers:

Execution Memory – what the agent has done before

- Past instructions and how they were decomposed
- Which approaches worked, which failed, and why
- Timing and cost of past executions
- The agent must demonstrably use this when it receives a similar instruction again

Capability Memory – what the agent knows how to do

- The tools and API operations it has used
- Success rates per operation
- Constraints discovered at runtime (rate limits, permission boundaries, field validation rules)

You must be able to show the memory state before and after an execution and explain concretely what changed and why.

3 Capability Synthesis

The agent will encounter instructions it cannot execute with its current tools. When this happens, it must not simply fail — it must attempt to synthesise a new capability.

Specifically, when the agent identifies a capability gap (an operation it needs but doesn't have a tool for), it must:

- Reason about what the tool needs to do
- Generate and test the implementation
- If it works, register it for future reuse
- If it fails after N attempts, report clearly what it tried and why it couldn't proceed

You define what "synthesise" means in your implementation. It could be generating a new API call sequence, building a transformation function, composing existing tools in a new way, or generating code. The requirement is that it happens at runtime, not at design time, and that successful synthesis persists into capability memory.

4 Self-Learning Loop

The system must get demonstrably better over repeated use. You must implement at least one measurable feedback signal that drives improvement:

- Execution time for similar tasks decreases as the agent learns the optimal approach
- Failure rate drops as the agent learns platform constraints
- Decomposition quality improves as the agent learns which sub-task orderings succeed

- Tool selection improves as the agent tracks which capabilities work for which instruction patterns

On your walkthrough call, you must show a before/after comparison with real numbers. "It gets better" is not acceptable. "Task X took 4 API calls on first run and 2 on the fifth run because the agent learned Y" is.

05 Requirements Summary

✓ MUST HAVE

- Natural language instruction → platform execution, end to end
- Persistent memory with at least two distinct layers (execution + capability)
- Memory is actively used to change behaviour, not just logged
- Capability synthesis at runtime for at least one novel instruction type
- Self-learning with at least one measurable improvement signal
- Structured execution report returned after every run
- Partial failure handling — no silent half-completions
- Working demo: 3 instructions of increasing complexity, run live on the call

⊘ MUST NOT

- Hard-code tool implementations for every platform operation (the synthesis must be real)
- Use the platform's official automation features as a shortcut (no Zapier, no n8n's own AI nodes)
- Wipe memory between demo runs — the learning must persist

✧ NICE TO HAVE — *shows depth*

- **Multi-agent decomposition** — a planner agent hands subtasks to specialist agents
- **Memory compaction** — the agent summarises old memory rather than growing it infinitely
- **Confidence scoring** — the agent knows what it's unsure about and says so
- **Rollback** — the agent can undo what it did if instructed or if validation fails

06 What to Submit

A GitHub repository containing:

- **Working code** — runnable with a single command after `.env` setup
- **ARCHITECTURE.md** — one page maximum. Answer three questions only:
 - What does your memory system store, and why did you structure it that way?
 - How does capability synthesis work in your implementation?
 - What is your learning signal, and what does the agent do differently on run N vs run 1?
- **DEMO.md** — the three instructions you will run live on the walkthrough call, and what the agent is expected to do for each
- A **~15-minute video** demonstrating and explaining the system you have built, with real examples

07 Constraints

- Python is preferred; any LLM provider, any framework
- No constraint on which libraries or agent frameworks you use — but be prepared to justify every major dependency
- The platform must be real and the agent must make real API calls — no mocks in the demo
- You may use any existing tools or build on top of them (LangChain, LlamaIndex, Temporal, etc.)

08

On Partial Submissions

This is a deliberately hard assignment. We do not expect every candidate to complete all four requirements to production quality in five days. Partial submissions are accepted and will be evaluated — with one condition: what you built must work and must have depth.

A complete but shallow implementation scores lower than a partial but deep one. Two requirements done properly — with a real memory architecture and a genuine learning signal you can reason about — is a stronger submission than four requirements done cheaply. Do not spread yourself thin trying to tick every box.

If you ran out of time, say so in ARCHITECTURE.md. Tell us what you would have built next, also detailing the how and the why. That answer is part of the evaluation.

09

The Thing We're Actually Testing

Every part of this assignment has a cheap version and a real version.

	THE CHEAP VERSION	THE REAL VERSION
Memory	A vector database that stores past prompts and retrieves similar ones.	A system that extracts structured knowledge from past executions — what worked, what didn't, what constraints were discovered — and uses that knowledge to make different decisions.
Capability synthesis	A lookup table of API endpoints described to the LLM.	A system that, when confronted with a task it cannot do, reasons about what it needs, builds something, tests it, and registers it.
Learning loop	"We added more examples to the prompt."	A measurable change in behaviour that you can point to with numbers.

The differences are apparent. Build the real version.

All the best!